Prolog

Lists

Lists in Prolog

- The list is an essential data structure in Prolog
  - [1, 2, 3, 4]
  - [] is an empty list
  - [1, 2, "three"] is a list of three elements
  - [Head | Tail] represents the head and the rest of the list
  - [1, 2 | Tail] is a list made of at least two element

Head and Tail operator

```
?- [T | Q] = [1, 2, 3, 4].
T= 1,
Q= [2,3,4]
?- [1 | [2,3,4]] = L.
L= [1,2,3,4]
?- [1,2,3 | [4]] = L.
L= [1,2,3,4]
?- [T | Q] = [1].
T= 1,
Q= []
?- [T | Q] = [].
no
```

List construction

```
cons(X, Y, [X|Y]).

?- cons(1, [2,3,4], L).
L= [1,2,3,4]

?- cons(X, Y, [1,2,3,4]).
X= 1,
Y= [2,3,4]

?- cons(1, [2,3,4], [1,2,3,4]).
yes
```

Appending lists

```
append([],Y,Y).
append([A|B],Y,[A|W]) :- append(B,Y,W).

?- append([1,2], [3,4], L).
L= [1,2,3,4]
?- append(X, [3,4], [1,2,3,4]).
X= [1,2]
?- append([1,2], [3,4], [1,2,3,4]).
yes
```

Reversing lists, version 1

```
reverse([],[]).
reverse([H|T],L) :- reverse(T,LL), append(LL,[H],L).

?- reverse([1,2,3,4],L).
L= [4,3,2,1]

?- reverse(L,[1,2,3,4]).
L= [4,3,2,1]
```

Reversing lists, version 2

```
reverse([],L,L)-!.
reverse([H|T],L,R):- reverse(T,[H|L],R).

reverse(L,R) :- reverse(L,[],R).
```

Why the cut?

?- reverse(L,[1,2,3,4]).

Member of a list liste

```
member(X,[X|L]).
member(X,[Y|L]) :- member(X,L).
```

List length d'une liste

```
length([],0).
length([X|L],N) :- length(L,NN), N is NN+1.
```

Inserting in a list

```
insert(A,L,[A|L]).
insert(A,[X|L], [X|LL]) :- insert(A,L,LL).

?- insert(c, [a, b], L).
L = [c, a, b] ;
L = [a, c, b] ;
L = [a, b, c] ;
no
```

Inserting in a liste

```
?- insert(a, L, [b, a, d, a, f]).
L = [b, d, a, f] ;
L = [b, a, d, f] ;
no
```

insert and delete are complementary!

Deleting elements of a list

```
delete(R,[R|L],L).
delete(R,[X|LL], [X|L]) :- delete(R,LL,L).
```

Deleting elements of a list

```
deleteall(X,[],[]).
deleteall(X,[X|T],Result) :-
    deleteall(X,T,Result).
deleteall(X,[H|T],[H|Result]) :- X\=H,
    deleteall(X,T,Result).

?- deleteall(2,[1,2,4,3,2,6,2,2],L).
L = [1, 4, 3, 6].
```

List intersection

```
intersection( [], Ys, [] ).
intersection( [ X | Xs ], Ys, [ X | Zs ] ) :-
            member( X, Ys ), !,
            intersection( Xs, Ys, Zs ).
intersection( [ X | Xs ], Ys, Zs ) :-
            intersection( Xs, Ys, Zs ).
```

What happens if the cut is removed?

Quicksort

```
sort([],[]).
sort([P|Q],T) :- partition(P,Q,G,D),
                 sort(G,GG), sort(D,DD),
                 append(GG,[P|DD],T).
partition(P,[X|L],[X|PG],PD) :- X < P,
                                partition(P,L,PG,PD).
partition(P,[X|L],PG,[X|PD]) :- X >= P,
                                partition(P,L,PG,PD).
partition(P,[],[],[]).
```

Repetitive operations in a list

- Trivial case

  task([], []). % 1st list: input, 2nd list: output

- Perform the task on one element
  - Possibly few cases
  - The rest is performed by the recursive definition

```
task( [ X | Xs ], [ X | Zs ] ) :-   % possibly more than one list as input
                test( X), !,        % if yes, X will be included in the solution
                task( Xs, Zs ).  % recursion
task( [ X | Xs ], Zs ) :-           % case where test has failed, X not in the solution
                task( Xs, Zs ).  % recursion
```

- Processing the elements of a list

```
process-list([],[]).
process-liste([X|L],[Y|T]) :- process(X,Y),
                              process-list(L,T).
```

Accumulating a result

```
sum(L,S) :- sum(L,0,S).
sum([X|L],T,S) :- TT is T+X, sum(L,TT,S).
sum([],S,S).
```